

FINAL DEVELOPMENT INSTRUCTIONS — CRM DATA QUALITY & ENRICHMENT PLATFORM

Use the complete original CRM Data Quality & Enrichment Platform specification already provided as the authoritative project specification.

ALL original requirements remain in scope, including:

- Python
- Django
- Django REST Framework
- PostgreSQL
- REST API design
- Authentication
- JWT
- Authorization
- RBAC
- Multi-tenancy
- Security
- Company management
- Contact management
- Filtering
- Sorting
- Pagination
- PostgreSQL optimization
- Django ORM optimization
- Transactions
- Concurrency
- CSV bulk imports
- Large-data processing
- Celery
- Redis
- Idempotency
- Validation
- Normalization
- Duplicate detection
- Data enrichment
- Data-quality scoring
- Audit logging
- Rate limiting
- File security
- Caching
- Dashboard statistics
- Asynchronous exports

- OpenAPI/Swagger
- Error handling
- pytest
- Integration/API/security testing
- Performance testing
- Observability
- Health/readiness checks
- Git
- Distributed-system concepts
- Large-data-storage concepts
- Snowflake concepts
- ETL/ELT
- OLTP vs OLAP
- Optional AI integration
- Docker
- CI/CD
- AWS architecture

However, modify the implementation strategy according to the following rules.

===== 1. MOST IMPORTANT CHANGE — LOCAL FIRST
=====

The entire application must FIRST be developed and tested directly on my local Windows development machine.

DO NOT use Docker during initial application development.

DO NOT use Docker Compose for PostgreSQL.

DO NOT run Django inside Docker.

DO NOT run PostgreSQL inside Docker.

DO NOT introduce AWS/GCP deployment while building the core application.

DO NOT introduce cloud-hosted databases during development.

Initial environment:

Windows | +-- Python 3.12+ | +-- Python Virtual Environment | | +-- Django | +-- Django REST Framework
| +-- pycpg | +-- pytest | +-- Other Python dependencies | +-- PostgreSQL installed locally | | +--
PostgreSQL Server | +-- psql | +-- pgAdmin | +-- Redis locally ONLY when background processing is
introduced later

The initial application architecture is:

Client | v Django REST API | v Django ORM | v Local PostgreSQL

This must work completely before infrastructure/containerization work begins.

===== 2. LOCAL POSTGRESQL =====

PostgreSQL must be installed directly on Windows.

Use the locally running PostgreSQL server.

Teach me how to:

- Verify PostgreSQL installation
- Use psql
- Use pgAdmin
- Connect to PostgreSQL
- Create the project database
- Create the application database user
- Grant appropriate permissions
- Inspect schemas
- Inspect tables
- Execute SQL manually
- Inspect indexes
- Run EXPLAIN
- Run EXPLAIN ANALYZE
- Inspect active connections
- Understand transactions and locks

Django must connect directly to this local PostgreSQL database using psycopg.

Do NOT use SQLite as the real project database.

Do NOT use Docker PostgreSQL.

===== 3. PYTHON LOCAL ENVIRONMENT =====

Use a normal Python virtual environment.

Teach:

```
python --version
```

```
python -m venv .venv
```

Activating the virtual environment on Windows/Git Bash/PowerShell as appropriate.

Teach:

pip requirements package installation dependency management virtual environments

Never install project dependencies globally unless there is a specific reason.

===== 4. ENVIRONMENT CONFIGURATION =====

Use environment variables.

Create:

.env.example

Do NOT commit:

.env

Examples:

DJANGO_SECRET_KEY DEBUG DATABASE_NAME DATABASE_USER DATABASE_PASSWORD DATABASE_HOST
DATABASE_PORT

Later:

REDIS_URL

and eventually cloud-related variables.

Never hardcode credentials.

===== 5. KEEP ORIGINAL ENTERPRISE ARCHITECTURE =====

Do NOT simplify the application into a basic CRUD tutorial.

Keep the original modular architecture:

config/ accounts/ organizations/ companies/ contacts/ imports/ validation/ enrichment/ audit/ common/
docs/

Keep business logic out of views where practical.

Use Django idioms rather than blindly reproducing Java architecture.

Explain why Django architecture differs from:

Spring Boot Controller Service Repository JPA/Hibernate

when relevant.

===== 6. KEEP MULTI-TENANCY
=====

Multi-tenancy remains mandatory.

Organization A:

Users Companies Contacts Imports

Organization B:

Users Companies Contacts Imports

Organization A must NEVER access Organization B's data.

Tenant filtering must happen in backend/queryset logic.

Do not trust organization_id supplied by clients.

Test tenant isolation.

Teach IDOR/BOLA.

===== 7. KEEP COMPLETE SECURITY
IMPLEMENTATION =====

Security remains a major part of the project.

Implement:

JWT authentication Access tokens Refresh tokens Secure password hashing Token expiration Refresh-token handling RBAC DRF permissions Tenant isolation Input validation Rate limiting File-upload validation Secure error handling Environment secrets

Teach and demonstrate protections against:

SQL injection XSS CSRF CORS misconfiguration Broken authentication Broken authorization IDOR/BOLA
Brute-force attempts Sensitive-data exposure Unsafe file uploads Oversized requests

Do NOT implement custom cryptography.

===== 8. KEEP RBAC
=====

Roles:

ADMIN MANAGER ANALYST

Use proper DRF permission classes.

Teach:

Authentication vs Authorization

401 vs 403

Role-based authorization

Object-level authorization

Tenant-level authorization

===== 9. KEEP POSTGRESQL DEEP LEARNING
=====

Do not merely use PostgreSQL as storage.

Use this project to deeply teach:

Primary keys Foreign keys Unique constraints Check constraints Indexes Composite indexes B-tree concepts
JOINS Aggregations Transactions ACID Isolation Locks Race conditions JSONB Query plans EXPLAIN EXPLAIN
ANALYZE Bulk inserts Bulk updates Efficient pagination Connection concepts

For every important index explain:

WHY it exists.

WHICH query benefits.

WHAT the write/storage trade-off is.

===== 10. KEEP DJANGO ORM PERFORMANCE
=====

Teach and use:

filter() get() exclude() exists() values() values_list() select_related() prefetch_related() only() defer()
bulk_create() bulk_update() iterator()

Create an intentional N+1 problem.

Measure/explain it.

Then fix it.

Compare with JPA/Hibernate where useful.

===== 11. KEEP LARGE DATA REQUIREMENTS
=====

The system should be designed to realistically handle:

10,000 100,000 500,000 1,000,000+

contacts.

Do not load huge datasets entirely into memory.

Teach:

Generators Iterators Streaming Chunk processing Batch processing Memory efficiency Bulk database
operations Transaction boundaries Backpressure concepts

===== 12. KEEP CSV IMPORT
=====

Keep:

POST /api/v1/imports/

and:

GET /api/v1/imports/{id}/

Keep ImportJob.

Statuses:

PENDING PROCESSING COMPLETED PARTIALLY_COMPLETED FAILED

Initially develop the data-processing logic locally.

When Celery is introduced later, move heavy processing into background workers.

===== 13. CELERY + REDIS — LOCAL, BUT LATER
=====

Celery and Redis remain part of the project.

However:

DO NOT introduce them in Phase 1.

First understand Django + PostgreSQL + REST + security.

When background processing becomes necessary, install/run Redis locally.

Then introduce Celery.

Architecture:

Django | v Local Redis | +---- Celery Worker 1 | +---- Celery Worker 2 | +---- Celery Worker 3 | v Local
PostgreSQL

Teach:

Producer Broker Queue Worker Task Retry Failure handling At-least-once delivery Idempotency Dead-letter
concepts

===== 14. KEEP IDEMPOTENCY
=====

Background tasks must tolerate retries.

Explicitly demonstrate:

Worker starts processing.

Worker partially completes work.

Worker crashes.

Task retries.

Data must NOT become duplicated or corrupted.

Use appropriate:

Unique constraints Job states Transactions Idempotency logic Safe retry logic

===== 15. KEEP DATA QUALITY PIPELINE
=====

Keep:

Contact | v Normalization | v Validation | v Duplicate Detection | v Enrichment | v Quality Score

Implement deterministic/local approaches first.

Do not depend on paid external APIs.

===== 16. KEEP NORMALIZATION
=====

Implement:

Email normalization Phone normalization Company normalization Job-title normalization

Example:

Sr. Software Eng.

->

Senior Software Engineer

Example:

MICROSOFT CORPORATION Microsoft Corp. microsoft

->

canonical Microsoft representation

===== 17. KEEP DUPLICATE DETECTION
=====

Level 1:

Normalized email

Level 2:

Phone

Level 3:

Normalized name + company

Optional:

Fuzzy matching

Teach:

Precision Recall False positives False negatives

Do not automatically merge uncertain matches.

===== 18. KEEP ENRICHMENT PROVIDER
ARCHITECTURE =====

Keep an enrichment-provider abstraction.

Example:

EnrichmentProvider

```
|  
+-- MockCompanyProvider  
|  
+-- MockContactProvider
```

Local application must not require paid enrichment APIs.

Design it so real providers can be introduced later.

===== 19. KEEP DATA QUALITY SCORING
=====

Example:

Valid email +25 Valid phone +20 Verified company +20 Normalized job title +15 Employment verified +20 ----
100

Keep scoring centralized/configurable.

Do not scatter magic numbers throughout code.

===== 20. KEEP TRANSACTIONS AND CONCURRENCY
=====

Teach:

transaction.atomic()

rollback

race conditions

select_for_update()

unique constraints

pessimistic locking

optimistic approaches conceptually

Create at least one realistic race-condition example and fix it properly.

===== 21. KEEP CACHING — LATER
=====

Caching remains part of the project.

Do NOT introduce Redis caching until the underlying PostgreSQL queries have been implemented and understood.

First:

Implement query.

Measure query.

Optimize PostgreSQL.

Then, if justified:

Cache it.

Teach:

Cache hit Cache miss TTL Cache invalidation Cache-aside Stale data Cache stampede

===== 22. KEEP TESTING
=====

Use:

pytest pytest-django

Test:

Authentication JWT RBAC Tenant isolation Contact CRUD Company CRUD Validation Normalization Duplicate detection Quality scoring CSV imports Malformed input Database constraints Transactions Concurrency-sensitive logic Celery retries Idempotency Security

Teach comparisons with:

JUnit Mockito Spring Boot tests

when useful.

===== 23. KEEP PERFORMANCE TESTING
=====

Generate synthetic datasets:

10K 100K 500K 1M

contacts.

Measure:

Listing Filtering Search Aggregation Imports

Show:

BEFORE optimization

and:

AFTER optimization

Use real measurements.

Never fabricate performance results.

===== 24. KEEP OBSERVABILITY
=====

Implement:

Structured logging Request IDs/correlation IDs where useful Job IDs Processing duration Error logging

Teach:

Logs Metrics Traces Alerts

Do not log:

Passwords JWTs Secrets Sensitive data unnecessarily

===== 25. KEEP HEALTH CHECKS
=====

Create:

GET /health/

GET /ready/

Teach:

Liveness vs readiness.

Initially readiness should verify local PostgreSQL.

Later it can verify Redis.

===== 26. KEEP GIT
=====

Use Git professionally throughout development.

Repository already exists.

Do NOT reinitialize Git.

Use meaningful commits.

Where practical use branches such as:

feature/authentication feature/contact-management feature/company-management feature/csv-import
feature/data-validation feature/background-processing feature/security feature/performance

Teach:

Branch Commit Merge Pull Request Rebase Cherry-pick Conflict resolution

Never automatically push unless I explicitly request it.

===== 27. DOCKER — MOVE TO FINAL STAGE
=====

Docker remains a learning objective.

However:

DO NOT use Docker while initially developing the application.

DO NOT create Docker Compose as part of Phase 1.

Only introduce Docker AFTER the complete application works locally.

Then teach:

Dockerfile Image Container Volume Network Docker Compose

Then containerize:

Django PostgreSQL Redis Celery

This phase exists to teach deployment/containerization, not to hide local development infrastructure.

===== 28. CI/CD — AFTER DOCKER
=====

CI/CD remains required but comes later.

After the application and tests are mature, create CI.

Pipeline can include:

Install dependencies Formatting Linting Tests Security/basic checks Build artifact/container

Teach:

Continuous Integration

vs

Continuous Delivery/Deployment

===== 29. AWS — FINAL PHASE
=====

AWS remains part of the project because cloud knowledge is relevant to the target role.

However:

DO NOT deploy to AWS during core development.

DO NOT create paid resources automatically.

After local application + testing + Docker + CI/CD are understood, design production architecture.

Possible final architecture:

Internet | v Route53 | v Load Balancer | v Django Application | +---- RDS PostgreSQL | +---- ElastiCache
Redis | +---- S3 | +---- Celery Workers | +---- CloudWatch

Teach:

EC2 ECS Fargate RDS S3 IAM Load Balancer Auto Scaling CloudWatch Secrets Manager

Explain trade-offs.

Do not incur cloud costs unless I explicitly approve deployment.

===== 30. SNOWFLAKE — KEEP AS OPTIONAL
LEARNING =====

Do not install Snowflake locally.

Do not make Snowflake required for the application.

Teach architecture:

PostgreSQL | v ETL / ELT | v Snowflake | v Analytics / BI / AI

Teach:

OLTP OLAP ETL ELT Data warehouse

Explain why PostgreSQL serves operational workloads while Snowflake is typically used for analytical workloads.

===== 31. OPTIONAL AI — KEEP
=====

AI remains an optional advanced feature.

Possible architecture:

AIProvider

```
+-- MockAIProvider
+-- OpenAIProvider
+-- GeminiProvider
```

Potential uses:

Job-title normalization Persona classification Company classification Anomaly explanation

The application must work without paid AI credentials.

Teach:

Hallucination Structured output Validation Latency Cost Privacy Retries Fallbacks

Never blindly trust AI output.

===== 32. UPDATED IMPLEMENTATION PHASES
=====

Follow THIS order.

PHASE 1 — LOCAL FOUNDATION

Python 3.12+ Virtual environment Django Django REST Framework Local PostgreSQL psycopg Environment configuration Django project structure Basic health endpoint

NO Docker.

NO Redis.

NO Celery.

NO AWS.

PHASE 2 — DATABASE FOUNDATION

PostgreSQL database/user Django ORM Models Migrations Constraints Relationships Basic SQL psycopg2 pgAdmin

PHASE 3 — SECURITY FOUNDATION

Organization Custom User model if justified Multi-tenancy JWT Authentication Refresh tokens RBAC DRF permissions Tenant isolation Security tests

PHASE 4 — CORE DOMAIN

Companies Contacts CRUD Serializers ViewSets REST design Filtering Sorting Pagination Swagger/OpenAPI

PHASE 5 — POSTGRESQL + ORM ENGINEERING

Indexes Composite indexes Query plans EXPLAIN ANALYZE N+1 select_related prefetch_related Bulk operations Transactions Locks Concurrency

PHASE 6 — DATA QUALITY

Normalization Validation Duplicate detection Quality scoring Enrichment provider abstraction

PHASE 7 — LARGE DATA

CSV upload ImportJob Streaming Chunk processing Batching Bulk insert/update Memory optimization Performance measurements

PHASE 8 — BACKGROUND PROCESSING

Install/run Redis locally

Install Celery

Workers Queues Retries Failure handling Idempotency

Move heavy imports/validation/enrichment into workers.

PHASE 9 — ENTERPRISE FEATURES

Audit logging Exports Dashboard Caching Rate limiting File security Centralized errors Observability Health/readiness

PHASE 10 — TESTING + PERFORMANCE

Unit tests API tests Integration tests Security tests Large-data tests Synthetic dataset generation Query benchmarks Import benchmarks

PHASE 11 — OPTIONAL ADVANCED DATA/AI

AI provider abstraction Mock AI Optional OpenAI/Gemini Snowflake architecture ETL/ELT concepts OLTP vs OLAP

PHASE 12 — DOCKER

ONLY NOW introduce Docker.

Dockerize:

Django PostgreSQL Redis Celery

Teach Docker thoroughly.

PHASE 13 — CI/CD

Create CI pipeline.

Run:

Linting Tests Security checks Build

PHASE 14 — AWS / PRODUCTION ARCHITECTURE

Design:

Load balancing Django scaling RDS ElastiCache S3 IAM Workers Auto Scaling CloudWatch Secrets Manager

Deployment only with explicit approval.

===== 33. LEARNING STYLE
=====

I am an experienced Java/Spring Boot developer.

For every important concept explain:

1. WHAT is this?
2. WHY do we need it?
3. HOW does it work?
4. HOW does Django/Python implement it?
5. WHAT is the equivalent or closest concept in Java/Spring Boot/JPA?
6. WHAT are common mistakes?
7. WHAT could an interviewer ask about it?

Examples:

Django Model $\approx$ JPA Entity

Django ORM $\approx$ Hibernate/JPA

DRF Serializer $\approx$ DTO + validation + mapping concepts

DRF ViewSet $\approx$ REST controller conceptually

Middleware $\approx$ Servlet Filter / Spring interceptor in some use cases

Celery $\approx$ Background/distributed job processing architecture

Do not force comparisons when they are misleading.

===== 34. DO NOT GENERATE EVERYTHING AT ONCE
=====

This rule is mandatory.

Before each phase:

Explain architecture.

Explain concepts.

List files to create/change.

Explain database impact.

Explain security impact.

Explain interview relevance.

THEN implement that phase.

After each phase:

Run application.

Run migrations.

Run tests.

Verify endpoints.

Fix errors.

Explain important code.

Update documentation.

Update interview notes.

Give me 5-10 interview questions.

STOP.

Wait for me before starting the next major phase.

===== 35. DOCUMENTATION
=====

Maintain:

README.md

docs/architecture.md

docs/interview-notes.md

README should eventually contain:

Project overview Technology stack Local Windows setup Python setup PostgreSQL setup Environment variables Running migrations Starting Django Starting Redis/Celery when introduced Testing API documentation Security architecture Database architecture Large-data architecture Performance considerations Docker setup later AWS architecture later

===== 36. IMPORTANT ENGINEERING RULES
=====

Never fabricate test results.

Actually run tests when possible.

Never hardcode credentials.

Never commit .env.

Never commit secrets.

Never disable security just to make something work.

Never expose another organization's data.

Never trust organization IDs from clients.

Never load huge CSV files entirely into memory.

Avoid N+1 queries.

Do not add indexes without understanding why.

Use database constraints where appropriate.

Use transactions where consistency matters.

Make background tasks idempotent.

Use timezone-aware timestamps.

Validate external input.

Use proper HTTP status codes.

Do not expose stack traces or sensitive information.

Do not blindly trust AI output.

Do not over-engineer.

Prefer understandable production-quality solutions.

===== 37. FINAL LEARNING TARGET
=====

At completion I should confidently understand and explain:

Python Django Django REST Framework PostgreSQL Django ORM REST APIs Authentication JWT
Authorization RBAC Multi-tenancy API security Database design Indexes Composite indexes EXPLAIN
ANALYZE Transactions Concurrency Locks Query optimization N+1 Pagination JSONB Large-data processing
Streaming Batching CSV processing Celery Redis Queues Workers Retries Idempotency Caching Testing
Performance testing Audit logging Observability Data validation Data normalization Duplicate detection
Data enrichment Data quality ETL ELT OLTP OLAP Snowflake basics AI integration Git Docker CI/CD AWS
Distributed systems Scalability

===== START NOW
=====

IMPORTANT:

DO NOT WRITE APPLICATION CODE YET.

First inspect the existing repository.

Then provide:

1. Proposed architecture
2. Django project/app structure
3. PostgreSQL database/entity design
4. Entity relationships
5. REST API endpoint list
6. Authentication architecture
7. RBAC/authorization architecture
8. Multi-tenancy architecture
9. Security architecture
10. Data-quality pipeline
11. Large-data strategy
12. Future Celery/Redis architecture
13. Testing strategy
14. Local Windows development requirements
15. Python dependency list with explanation
16. Updated implementation phases
17. Key architectural trade-offs
18. First 20 interview concepts this project will teach me

Remember:

DEVELOP LOCALLY FIRST.

Use:

Python + Django + Django REST Framework + LOCAL PostgreSQL.

NO DOCKER IN INITIAL DEVELOPMENT.

NO AWS IN INITIAL DEVELOPMENT.

NO REDIS/CELERY UNTIL THE BACKGROUND-PROCESSING PHASE.

Docker, CI/CD and AWS come AFTER the application works locally.

After presenting the architecture, STOP and wait for my approval before implementing Phase 1.